Part 1: Module reflection and Summary

Reflection

What is the most important thing you learnt in this module?

I'd say some important things in this module was how networks communicate with each other as a complete end to end process, through the five layer internet protocol stack. Apart from just explaining and understanding protocols like TCP, UDP, DNS, HTTP and Ethernet individually, this module shows how these protocols work in each layer, encapsulating and decapsulating data. The wireshark activities made this way more practical by allowing me to inspect real world DNS, TCP, UDP, ARP and HTTP packets. This was my first time comparing wireshark packet capture results to the information displayed in chrome dev tools. It was practical, engaging and fun to do.

How does this relate to what you already know?

This module was more like a surface level recap of networking concepts that I'm already familiar with. Previously, I have read Comptia's Network+ study guide, and Cisco's CCNA Volume 1 (Network Fundamentals). I also already have practical experience with linux networking, routing, NAT, DHCP, DNS, wireshark and deploying and setting up properly segmented networks for work contacts that I've done before (Ubiquiti networking equipment are my favorite for this). Even circuit switching was covered to a similar level during my final high school examinations (CIE A/L Computer Science) – which I have also tutored students for. However, revisiting these concepts helped me reinforce the theoretical foundations behind what I already deal with in practice. I'm more interested and looking forward for the future modules of this unit, particularly, stuff involving routing protocols, AQM related things, network security, traffic analysis and more lower level concepts.

Why do you think your course team wants you to learn the content of this module?

I think the main reason for this module's content is to ensure that students have a high level foundational understanding for more advanced networking concepts in the future modules. Understanding the layered internet protocol stack, routing, addressing, transport protocols and application level protocols / traffic is necessary for understanding how networks function as a whole. Having this core, foundational understanding will help us both secure networks more effectively and also to diagnose and troubleshoot network related issues better. For example, a website not loading could be caused by the HTTP, TCP, DNS, routing, ARP or maybe even the physical connection itself.

The module also introduced concepts like delay, throughput, packet loss, congestion, and many other related concepts which explain why a network might be performing poorly even when its "connected". Overall, it developed the packet level understanding required to analyze, troubleshoot, optimize and secure networks as a whole.

—> Next Page

Summary

The internet is a global network of interconnected networks. This allows distributed applications running on different systems to communicate with each other. Devices connected to a network are called hosts or end systems or nodes. These include desktops, servers, phones, IoT devices and other end devices. These hosts communicate with each other through communication links, switches and routers. A link between a source and a destination is called a route or a path.

Network protocols define the format of data, the order in which the data should be exchanged, and what to do when this messages are received. They are basically a set of rules defining what to do with network data. Protocols allow devices from different manufacturers to be intercompatible with each other as well. They are basically standardized through IETF RFCs while some protocols/standards like Ethernet and WiFi are handled by IEEE.

The network edge has clients, servers and other end systems. Servers are what serves services to clients. Clients initiate a request and sends to a server, to which it responds with a response. Applications usually communicate through a socket interface, which is identified by the IP address, transport protocol and the port number.

When a client accesses a network, the end system first connects to to it's first router – which is sometimes also called the edge router. Access links might be either dedicated or shared between multiple users. Shared links might experience contention and lower per user throughput when many users transmit data simultaneously.

Network traffic is usually sent through twister pair copper cables, optical fiber cables and as radiowaves. Each medium of transfer has its own pros and cons relating to bandwidth, range, interference, attenuation and many other factors. This is all handled at the physical layer and it represents bits using electrical, optical or radio frequency signals.

Packet switching networks divides application data into smaller packets that can independently traverse shared network links. Routers usually use "store and forward" switching. Here, the packet must fully be received before its transmitted/forwarded to the next link. The routers usually use statistical multiplexing – this allows the bandwidth to be dynamically shared among multiple users.

Circuit switching on the other hand reserves a fixed amount of network capacity for itself for the entire duration of its session. Resources are usually divided using approaches like (orthogonal) frequency division multiplexing and time division multiplexing. Circuit switching provides a predictable capacity and a delay but the resources will be kept reserved even when they are not technically being used, this wasting them. Because of it's fixed nature, this is not that efficient for bursty traffic – which packet switching handles fairly smoothly. Packet swicthing is also generally more efficient for modern day to day traffic and web traffic.

The end to end network delay is made up of 4 main components.

- The processing delay: examining headers and selecting an outgoing interface
- Queuing delay: waiting in a router buffer
- Transmission delay: placing all packet bits onto the link
- Propagation delay: the signal traveling through the physical medium.

The transmission delay can be generalized and calculated as follows: **d = packet_size / link_rate**, and the propagation delay can be calculated as: **d = distance / signal_propagation_speed**. The queuing delay changes a lot and isn't really that important until the unless the link utilization is at a 100%.

A router has a limited capacity for its buffer. Therefore, if packets arrive faster than packets being forwarded out, they may accumulate in the output queue. And when this queue is full, the newly arriving packets will get dropped.

Throughput is the actual rate at which useful data is transferred between systems, and the end to end throughput is limited by the “bottleneck link” – which is the link with the lowest capacity in the path.

Active Queue Management operates around the output queue of a router or a network interface. Instead of just waiting for the buffer to get full, AQL will drop selected packets, ECN mark packets and does takes a lot of other proactive measures. Algorithms like RED, CoDel, PIE, CAKE, and more commonly DualPI2 are designed to control the queuing delay and to reduce the buffer bloat. More recently, there has been research into making L4S AQM better using LLMs/ML models[1] in a very performant and smart way.

The internet protocol stack has 4 layers which works with each other.

Application Layer:

- This layer provides network services directly to applications and software.
- Some protocols at this layer are: HTTP/HTTPS for web communication, DNS for hostname resolution, SSH for remove administration, SMTP for email transmission, and DHCP for network configuration.
- For example, when I try to access a website, my browser will generate an HTTP request at the application layer. A very basic HTTP request would look something like GET / HTTP/1.1. But of course, other values may also be passed in along with this.

Transport Layer:

- This layer provides end to end communication between processed running on multiple hosts. It uses post numbers (ranging from 1-65535 (calculated with $2^{16}-1$) to identify the specific source and destination applications. The two main protocols at this layer are TCP and UDP.
- TCP (Transmission Control Protocol):
    - Each data unit is called a TCP segment.
    - This is a connection oriented protocol that allows for an ordered full duplex byte stream communication between applications. It uses sequence numbers, acknowledgments, retransmissions with checksums, and congestion control algorithms. Each TCP connection is established with a three way handshake.
- UDP (User Datagram Protocol):
    - Each data unit is called a UDP datagram.
    - This is a connectionless protocol with low overhead. It doesn’t establish a connection or a “session” or provide acknowledgments, retransmissions, flow control, congestion control or any other “connection oriented” features. Each datagram is transmitted independently using source and destination ports with checksums for integrity checking. Due to this connectionless nature, UDP provides a very low latency and is therefore used in DNS resolving, VoIP, online gaming, streaming, and any other latency sensitive task.

Network Layer:

- This layer is responsible for logical addressing of devices, allowing the packets to be routed across different networks even. It will handle the src/dst IP addressing, routing, packet forwarding, managing hop limits & TTL and many other things.
- The main protocol at this layer is IPv4 or IPv6. Routers mostly work in this layer.

- For example, my GL.iNet router takes care of routing my traffic between my private home subnet and the landlord's network. For internet related IPv4 traffic, it will handles NAT, replacing the private src IP with an address reachable through it's WISP uplink.

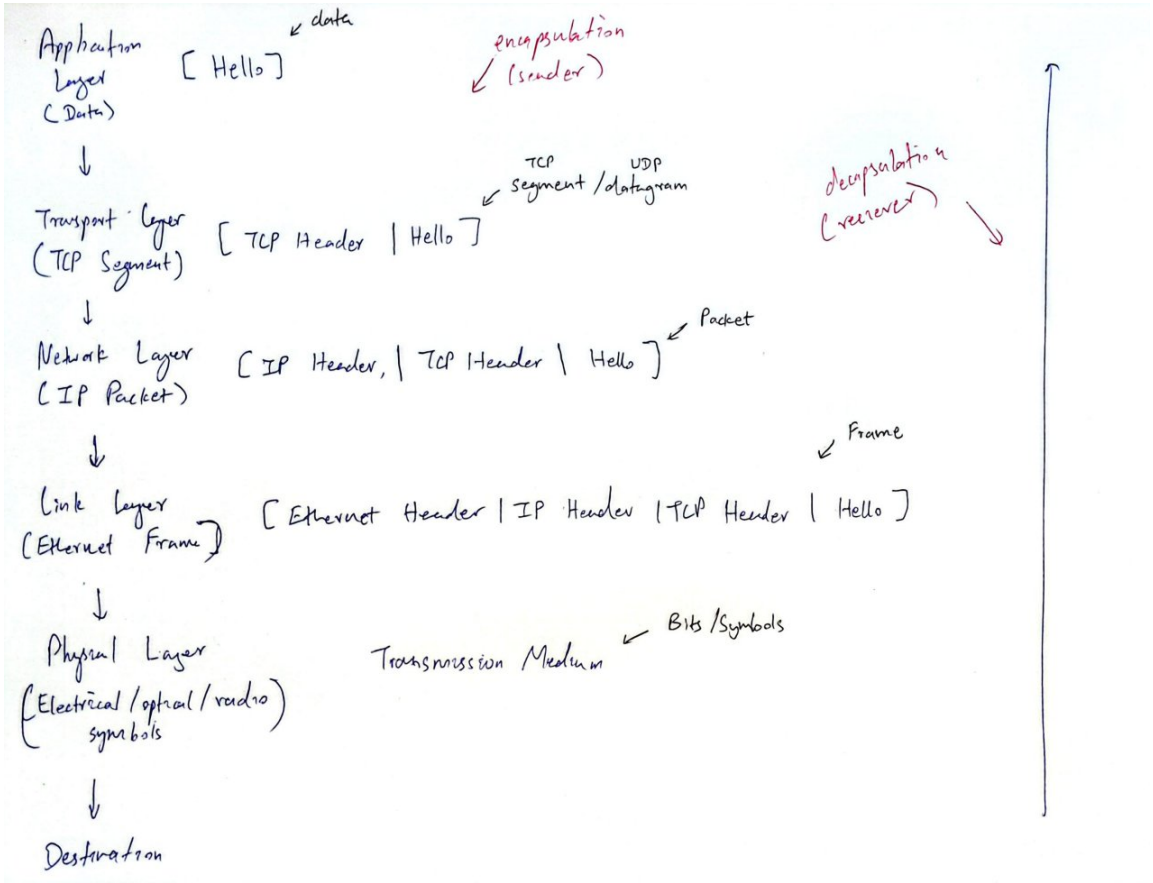
Link Layer:

- This handles the transfer of frames between devices on the same local network segment. It will deal with MAC addressing, creating frames, ensuing local delivery, and it also resolves the addresses of the next hop using ARP (for IPv4) or IPv6 neighbor discovery.
- Protocols used in this layer includes: Ethernet, IEEE 802.11 WiFi, ARP, and many more...
- For local communication between my wired desktop and my home server, the ethernet frames are basically just switched within the LAN. That traffic will not pass through NAT (or into another router) because both my devices are on the same private subnet.

Physical Layer:

- At this layer, frames are converted into electrical/optical/radio signals that can travel through the transmission medium of choice. This layer deals with modulation and encoding, connections, channels and physical media and also deals with frequencies, voltages and optical signaling.

The diagram below demonstrates how every layers works with each other, both encapsulating and decapsulating things as data is sent and received.



Chrome dev tools can be used to see network traffic, analyze page performance, see the page source and for general troubleshooting. This is useful for both gaining a fundamental networking understanding as well as actually developing and debugging websites. You can see all the network requests sent, the request methods, status codes, protocols being used, domain, response, timing, and basically a lot of other information. The waterfall view shows when resources are requested from the server, when they are done loading and when they are done being processed,

Wireshark is a packet capture tool that shows all the packets sent and received over a network interface. Some of the important fields of each captured packet includes the frame number, the timestamp, source & destination, the protocol, and other packet information. You can directly filter by the protocol, by typing things like `http`, `dns`, `tcp` or basically anything else that is supported. Below are some sample display filters:

- udp.port == 53 —> shows all UDP traffic sent over port 53
- ip.src == 192.168.8.136 —> shows all traffic with the source IP of 192.168.8.136

A list of all the available filters can be found here[2] in their official documentation. A quick guide to more commonly used filters and also their general syntax can be found here[3].

References

[1] D. Satish, P. S. Raj, J. Kua, and A. Walid, “Distilling Large Language Models for Network Active Queue Management,” arXiv.org, 2025. https://arxiv.org/abs/2501.16734 (accessed July 14, 2026).

[2] Wireshark, “Wireshark · Display Filter Reference: Index,” www.wireshark.org. https://www.wireshark.org/docs/dfref/ (accessed July 14, 2026).

[3] Wireshark, “DisplayFilters - The Wireshark Wiki,” Wireshark.org, 2017. https://wiki.wireshark.org/DisplayFilters (accessed July 14, 2026).